Recommendation Systems for Personalized Content Discovery

Abstract

This project develops personalized movie recommendation systems using the Netflix Prize Dataset containing over 100 million ratings from 480,189 users across 17,770 movies. Three recommendation approaches were implemented and compared: User-Based Collaborative Filtering, Item-Based Collaborative Filtering, and Matrix Factorization. Models were evaluated using RMSE for rating prediction accuracy and MAP@10 for recommendation ranking quality. The GPU-accelerated Matrix Factorization model achieved the best performance with a test RMSE of 0.8135 and MAP@10 of 0.04965, demonstrating the effectiveness of latent-factor methods for large-scale recommendation tasks.

1. Introduction

Recommendation systems help users discover relevant content from large catalogs and are widely used by platforms such as Netflix, Spotify, and Amazon. The objective of this project is to learn user preferences from historical ratings, predict unseen ratings, and generate personalized movie recommendations. Three approaches were implemented and evaluated: User-Based Collaborative Filtering, Item-Based Collaborative Filtering, and Matrix Factorization.

2. Dataset Description

The Netflix Prize Dataset contains explicit movie ratings collected from Netflix users. Each interaction contains a UserID, MovieID, Rating (1–5), and Date. Movie metadata including title and release year was also used for recommendation interpretation.

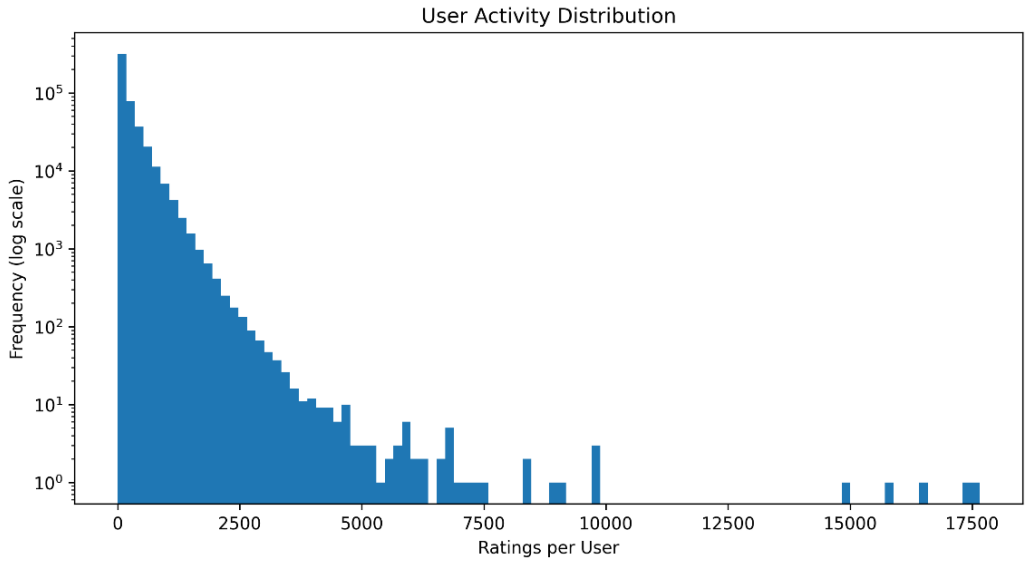
Attribute	Value
Users	480,189
Movies	17,770
Ratings	100,480,507
Rating Scale	1–5

The dataset is highly sparse and exhibits strong long-tail behavior, making it suitable for evaluating recommendation algorithms.

3. Exploratory Data Analysis

3.1 User Activity Analysis

Figure 1. User Activity Distribution



Key Findings

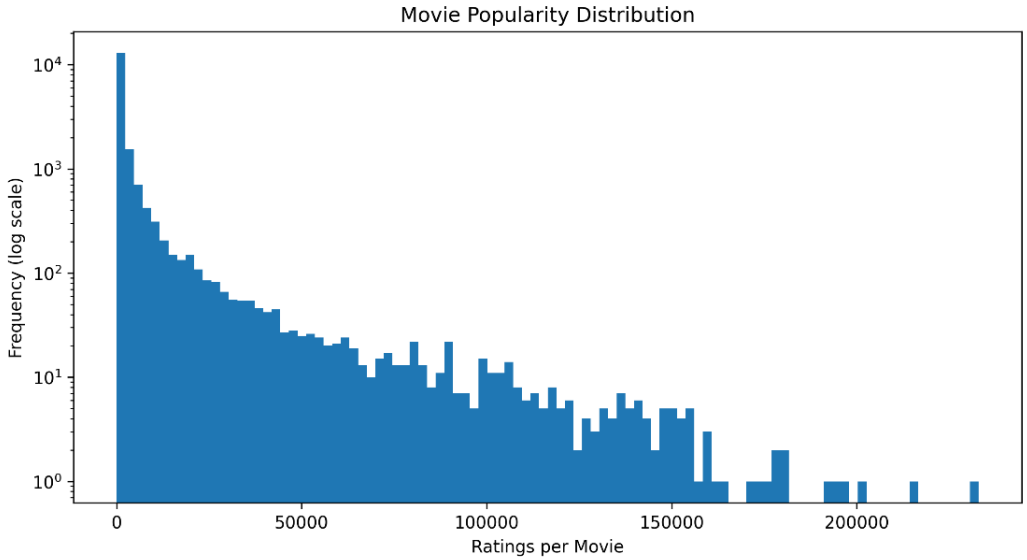
- Mean ratings per user: **209** | Median: **96** | Maximum: **17,653**
- User activity is highly skewed, with a small group of very active users contributing a large number of ratings.

Implication:

Sparse user histories make personalized recommendation challenging.

3.2 Movie Popularity Analysis

Figure 2. Movie Popularity Distribution



Key Findings

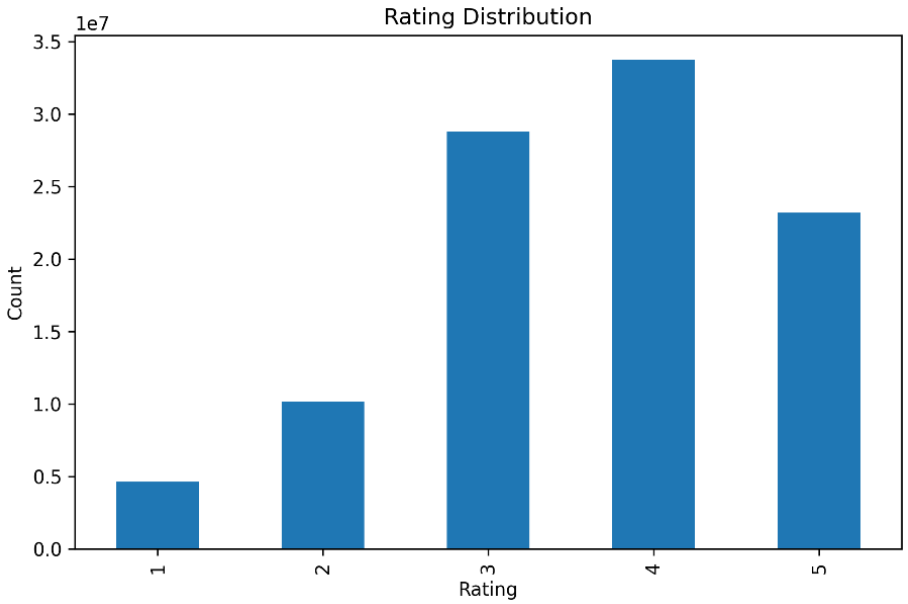
- Mean ratings per movie: **5,655** | Median: **561** | Maximum: **232,944**
- Movie popularity follows a strong long-tail distribution.

Implication:

Recommendation models may favor highly popular movies.

3.3 Rating Distribution

Figure 3. Rating Distribution



Key Findings

- Ratings are concentrated around **3–5 stars**; positive ratings dominate the dataset.

• Users tend to rate movies they enjoy — ranking relevant recommendations is as important as predicting exact ratings.

3.4 Sparsity Analysis

Metric	Value
Density	1.18%
Sparsity	98.82%

• Only a small fraction of possible user-movie interactions are observed.

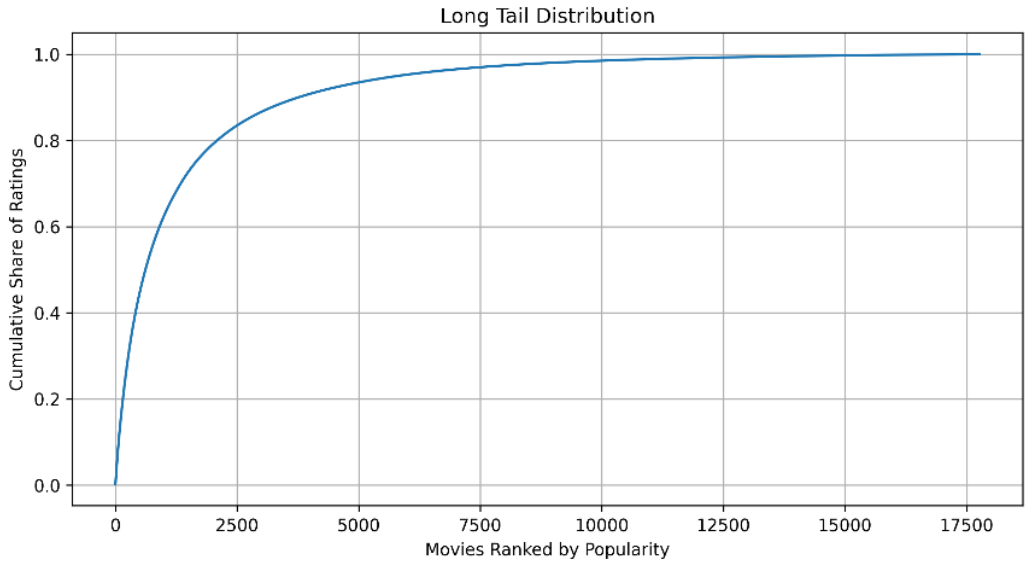
• Most entries in the user-item matrix are missing.

Implication:

Collaborative filtering and latent-factor methods are required to infer missing preferences.

3.5 Long-Tail Analysis

Figure 4. Long-Tail Popularity Curve



• A small number of movies receive most interactions.

• The majority of movies belong to the long tail with limited ratings.

Implication:

Effective recommendation systems should balance relevance and content diversity.

3.6 EDA Summary

The Netflix dataset exhibits: highly uneven user activity; strong long-tail movie popularity; predominantly positive ratings; extreme sparsity (98.82%); and potential popularity bias. These observations motivated the use of collaborative filtering and matrix factorization approaches.

---

4. Data Preparation and Experimental Setup

To reduce computational complexity while preserving sufficient interaction data, users with at least 500 ratings were selected for model development.

Metric	Value
Active Users	54,404
Ratings Used	47,092,231
Movies	17,770

A user-wise interaction split was applied: **80%** training, **10%** validation, **10%** test. The training set was used for model learning, validation for hyperparameter tuning, and the test set for final evaluation.

---

5. Baseline Models

Three simple baseline predictors were implemented to establish reference performance:

Model	Description
Global Mean	Overall average rating
User Mean	Average rating given by each user
Movie Mean	Average rating received by each movie

---

6. Collaborative Filtering Models

6.1 User-Based Collaborative Filtering

User-Based CF predicts ratings using preferences from similar users.

Methodology

- Construct user-item interaction matrix and compute cosine similarity between users.
- Identify Top-50 nearest neighbors and predict ratings using weighted neighbor ratings.

Advantages

- Easy to interpret; captures community-level preferences.

Limitations

- Computationally expensive for large datasets; performance degrades with sparse user histories.

6.2 Item-Based Collaborative Filtering

Item-Based CF recommends movies similar to those previously liked by a user.

Methodology

- Compute item-item cosine similarity and identify Top-50 similar items.
- Estimate ratings using weighted item similarities.

Advantages

- More scalable than User-CF; stable item relationships.

Limitations

- Limited ability to capture hidden user preferences; performance affected by sparse item interactions.

---

7. Matrix Factorization

Matrix Factorization was selected as the primary recommendation model due to its ability to learn latent user preferences and movie characteristics. The user-item interaction matrix is decomposed into low-dimensional user and movie embeddings:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$$

where: μ = global mean rating, b_u = user bias, b_i = item bias, p_u = user latent vector, q_i = movie latent vector.

Model Architecture

The model was implemented using PyTorch and consists of: user embedding layer, movie embedding layer, user bias embedding, movie bias embedding, and a global rating bias.

Training Configuration

Parameter	Value
Latent Factors	64
Optimizer	Adam
Batch Size	262,144
Epochs	15
Early Stopping	Patience 2
Device	GPU

Validation RMSE was monitored during training, and the best-performing model was selected for final evaluation.

Why Matrix Factorization?

Compared to neighborhood-based methods, Matrix Factorization handles sparse data effectively, learns hidden user-item relationships, scales better to large datasets, and produces more accurate rating predictions.

---

8. Recommendation Generation Pipeline

The recommendation process consists of four stages:

- **Candidate Generation:** Identify movies not previously rated by the user.
- **Rating Prediction:** Estimate ratings using the trained model.
- **Ranking:** Sort candidate movies by predicted score.
- **Top-K Selection:** Return the highest-ranked recommendations.

For evaluation, a movie was considered **relevant** if its actual rating ≥ 4. This threshold was used for MAP@10 computation and recommendation quality assessment.

---

9. Evaluation Methodology

RMSE

RMSE measures the difference between predicted and actual ratings and evaluates rating prediction accuracy. Lower RMSE indicates better prediction performance.

MAP@10

MAP@10 evaluates recommendation ranking quality. For each user, the Top-10 recommendations were generated and compared against relevant items (rating ≥ 4) in the test set. MAP@10 measures how effectively relevant movies are ranked near the top of recommendation lists.

---

10. Experimental Results

10.1 Rating Prediction Performance

Model	RMSE
Global Mean	1.0725
Movie Mean	0.9976
User Mean	0.9789
User-CF	1.0340
Item-CF	1.0137
Matrix Factorization	0.8135

Matrix Factorization achieved the lowest RMSE and significantly outperformed both collaborative filtering approaches and all baseline models.

10.2 Recommendation Performance

Metric	Value
MAP@10	0.04965
Coverage	9.45%

The model successfully generated personalized recommendations while maintaining reasonable catalog coverage.

---

11. Recommendation Analysis

Example User Recommendations

Based on previously liked movies — *Streets of Fire, Amadeus, Dangerous Liaisons, Moonlighting: The Pilot,* and *The Ninth Gate* — the model recommended:

Recommended Movies
Independence Day
The Silence of the Lambs
Pretty Woman
The Lord of the Rings: The Fellowship of the Ring
The Green Mile

The recommendations contain highly rated and widely appreciated movies that align with the user's historical preferences, demonstrating that the model successfully captures latent relationships between users and movies.

---

12. Coverage and Popularity Bias

Metric	Value
Coverage	9.45%
Average Popularity of All Movies	2,119
Average Popularity of Recommended Movies	8,081

Recommended movies were substantially more popular than the average movie in the dataset, indicating the presence of popularity bias in the recommendation output.

---

13. Limitations

The proposed system has several limitations:

- Cold-start users cannot be accurately modeled due to lack of interaction history.
- New movies receive limited exposure because of insufficient ratings.
- Recommendations exhibit popularity bias toward frequently rated movies.
- Ranking performance remains lower than rating prediction performance.

Future improvements may include hybrid recommendation systems, content-based features, and ranking-aware optimization objectives.

---

14. Conclusion

In this project, multiple recommendation approaches were implemented and evaluated using the Netflix Prize Dataset. User-Based Collaborative Filtering, Item-Based Collaborative Filtering, and Matrix Factorization were compared using RMSE and MAP@10. Among all methods, **Matrix Factorization achieved the best performance** with a test RMSE of 0.8135 and MAP@10 of 0.04965. The model successfully learned user preferences, generated personalized movie recommendations, and outperformed traditional neighborhood-based collaborative filtering methods. Overall, the results demonstrate the effectiveness of latent-factor recommendation models for large-scale personalized content discovery while highlighting challenges such as popularity bias and cold-start scenarios.